Orbit: MPC-Augmented Routing for Heterogeneous LLM Serving — Design, Evaluation, and Lessons Learned

Submitted Anonymously for Double-Blind Review

ABSTRACT

Heterogeneous LLM inference clusters — where servers differ in GPU generation, thermal state, or concurrency capacity — expose a gap in existing routers that treat all servers as identical. We design and implement **Orbit**, a request router that augments Power-of-Two-Choices (PO2) with a Model Predictive Control (MPC) layer that estimates per-server service rates and computes capacity-aware routing weights via a short-horizon quadratic program. We evaluate Orbit on AMD MI300X GPUs with five policies (Round Robin, Least Outstanding Requests, PO2, MPC-PO2, and a new Kalman-PO2) across a 1×–8× heterogeneity sweep and a 10,000-request convergence study. Our central finding is a *negative result*: both MPC-PO2 and Kalman-PO2 are consistently 15–40% worse than PO2 at steady state across all tested conditions, and the gap *widens* rather than closes under overload (4–12 rps). Kalman-PO2 shows a transient advantage in short runs (327 ms vs. 386 ms mean, 10× lower P99) by eliminating PO2's burst variance, but this benefit does not persist beyond ≈500 requests. An overload study (up to 12 rps, 3× baseline) falsifies the hypothesis that PO2 breaks down under load: LOR and PO2 remain the most robust policies throughout. We identify the root cause — both MPC and Kalman measure *experienced* load rather than *intrinsic* capacity, leaving no exploitable signal beyond what PO2 already captures — and derive concrete redesign directions. We present this as a cautionary case study in applying control-theoretic methods to LLM serving.

Keywords: LLM serving, request routing, load balancing, model predictive control, Kalman filter, heterogeneous inference, vLLM, ROCm, AMD MI300X, negative result

1 Introduction

GPU clusters serving LLMs exhibit significant heterogeneity in practice: servers run on different GPU generations, experience variable thermal throttling, hold different amounts of cached KV data, and operate under mixed tenant workloads. A router that ignores these differences will overload slower servers, inflating tail latency even when aggregate capacity is sufficient.

Reactive policies such as Least Outstanding Requests (LOR) and Power-of-Two-Choices (PO2) [3] respond to observed queue depth instantaneously but cannot, in principle, distinguish a server that is slow because it is overloaded from one that is intrinsically lower capacity. This motivates a predictive approach: if a router can *learn* each server's throughput, it can steer traffic proportionally to capacity rather than reactively to queue depth.

We present **Orbit**, which implements this idea by wrapping PO2 with a Model Predictive Control (MPC) [6] layer. Orbit estimates per-server service rates via exponential moving averages (EMA) and solves a quadratic program (QP) every 100 ms to compute routing weights that drive queue depths toward a target. The implementation integrates with AMD's vLLM ROCm stack and supports both standalone and prefill-decode disaggregated topologies.

We evaluate Orbit thoroughly and report an **honest negative result**: MPC-PO2 does not outperform PO2 or LOR under any tested condition, and the performance gap is persistent — it does not close with more traffic. We believe this result is more valuable than a selective positive report because it exposes *why* queue-depth-based MPC fails for LLM routing and what signals would actually work.

2 Design

2.1 System Architecture

Orbit is a FastAPI service between clients and a pool of vLLM servers. In *single-pool* mode, each request goes to one server performing both prefill and decode. In *PD-disaggregated* mode, the router selects a prefill and a decode server per request, injecting peer addresses into the request ID consumed by vLLM's KV-transfer connector. Four routing policies share the same infrastructure: **RR** (round-robin), **LOR** (min inflight count), **PO2** (sample two, pick lower weighted-inflight score), and **MPC-PO2** (PO2 with MPC-computed per-server weights).

2.2 MPC Weight Controller

Every $\Delta t = 100\,\text{ms}$, Orbit solves a per-server QP to update weights w_i. Let $\hat{\mu}_i$ be the EMA service rate ($\alpha = 0.3$), $\bar{q}$ the mean inflight count, and $q^* = 1.0$ the normalized target. The horizon-H QP is:

$$\min_{w,\,q} \quad \sum_{k=0}^{H-1} \left[(q_{k+1} - q^*)^2 + \lambda (w_k - 1)^2 \right] \tag{1}$$

$$\text{s.t.} \quad q_{k+1} = q_k + \Delta t \cdot \tfrac{\lambda_a p_i - \hat{\mu}_i}{\bar{q}}, \quad q_{k+1} \geq 0, \quad w_{\min} \leq w_k \leq w_{\max}$$

where λ_a is the arrival rate, $p_i = w_k / \sum_j w_j$ is server i's traffic fraction, and $\lambda = 0.07$ penalizes weight deviation. We use $H = 10$, $w_{\min} = 0.1$, $w_{\max} = 3.0$, solved with OSQP [7] in under 1 ms. The regularization $\lambda(w_k - 1)^2$ prevents extreme weights during estimation warm-up [6].

3 Evaluation

3.1 Experimental Setup

All experiments run on AMD MI300X nodes (192 GB HBM3/GPU) with vLLM v0.23.0 ROCm, serving Qwen3-8B in bfloat16 (`max-model-len=2048`, `gpu-memory-utilization=0.15`). Heterogeneity is simulated via two vLLM servers on separate GPUs: **Server 0 (fast):** `max-num-seqs=64` (fixed); **Server 1 (slow):** `max-num-seqs` $\in \{64, 32, 16, 8\}$, yielding 1×–8× capacity ratios. Requests follow Poisson arrivals at 4 req/s with 15 fixed prompts (`max_tokens=50`). The router is freshly restarted before each policy run so MPC starts with no prior EMA state.

3.2 Heterogeneity Sweep

Table 1 shows results for 120 requests across all four heterogeneity levels. **RR** shows high P99 (1775–2587 ms) from periodically routing to the slow server regardless of capacity. **LOR and PO2 perform comparably** across all ratios (mean 285–294 ms), with PO2's random sampling introducing minor noise. **MPC-PO2 is 14–20% worse** than PO2 at every heterogeneity level, with elevated stdev (42–50 ms vs. 17–20 ms for PO2). Critically, the MPC overhead is *independent* of the heterogeneity ratio — the same penalty at 1× as at 8× — suggesting the problem is not the degree of heterogeneity but the nature of the routing signal.

TABLE 1. Heterogeneity sweep: 4 rps, 120 req, fresh router per policy.

Hetero	Policy	Mean (ms)	Stdev (ms)	P99 (ms)
1×	RR	413.8	431.7	2587.1
	LOR	**292.3**	19.7	352.9
	PO2	289.0	17.8	323.5
	MPC-PO2	341.8	47.4	444.2
4×	RR	354.0	313.7	1828.4
	LOR	**285.2**	17.7	330.9
	PO2	285.9	19.7	344.4
	MPC-PO2	326.4	45.8	429.4
8×	RR	353.8	338.1	2417.1
	LOR	**287.9**	16.7	330.8
	PO2	287.9	17.8	342.4
	MPC-PO2	346.6	49.6	448.4

3.3 Convergence Study: 10,000 Requests

To test whether MPC eventually converges past a cold-start period, we ran PO2, MPC-PO2, *and* Kalman-PO2 for **10,000 requests** at 4 rps with 8× heterogeneity (≈42 minutes each), using fresh vLLM containers per policy. Table 2 shows per-200-request windows. The MPC gap *never closes*: MPC-PO2 is 13–22% worse than PO2 from the first window to the last, with no downward trend. Kalman-PO2 similarly settles at 15–21% above PO2 throughout, confirming that the advantage seen in the 500-request pilot (§5) does not persist to steady state.

TABLE 2. Convergence study (8× hetero, 4 rps, 10 K req). Selected windows of 200 requests; full 50-window table in accompanying repository.

Req window	PO2 (ms)	MPC (ms)	Kalman (ms)	MPC vs PO2
0–200	537.9	418.8	612.4	**+22.1%**
200–400	289.7	330.7	325.3	−14.2%
1000–1200	288.2	336.7	332.7	−16.8%
3000–3200	289.3	331.7	331.9	−14.7%
5000–5200	288.4	332.5	342.7	−15.3%
7000–7200	282.9	330.5	336.3	−16.8%
9800–10000	282.9	335.6	324.4	−18.6%
Overall	**291.8**	**338.2**	**342.9**	**−15.9%**

3.4 Root Cause Analysis

The convergence failure reveals a fundamental mismatch between the MPC design and the routing problem. At 4 rps with 8× heterogeneity, our log analysis shows PO2 routes 49%/51% across the two servers by request count, yet achieves mean latency of 289 ms — almost identical to LOR. This is because PO2 *routes opportunistically*, sending short requests to whichever server happens to be available. The slow server's higher latency means it is sampled as higher-load more often, so PO2 naturally deflects the longest-running requests away from it. The EMA rate estimator, operating on a 300 ms window at 4 rps, sees fewer than 1 completion per server per window — insufficient to distinguish a slow server from a temporarily busy fast server. There is effectively no exploitable signal for MPC.

MPC's overhead then makes things worse: the regularization term $\lambda(w_k - 1)^2$ biases weights toward 1.0, occasionally sending extra traffic to the slow server that PO2 would have deflected.

3.5 Overload Study: 4–12 rps

One might expect that PO2's implicit balancing breaks down under high load, where queues saturate and queue-depth ceases to be informative. We test this directly by sweeping arrival rates from 4 to 12 rps (3× above baseline) at 8× heterogeneity, 500 requests per cell, fresh containers per policy. Table 3 shows the results.

The overload hypothesis is **falsified**: MPC-PO2 and Kalman-PO2 do not recover under load — they get *worse*. At 12 rps, Kalman-PO2 is 39% above PO2 in mean latency. A 2,000-request convergence run at 12 rps confirms the gap is persistent: PO2=376 ms, MPC-PO2=421 ms (+12%), Kalman-PO2=485 ms (+29%), with no downward trend across ten 200-request windows.

Two additional observations: (1) **LOR emerges as the most robust policy under overload** — it ties or beats PO2 at 8–12 rps while MPC/Kalman degrade. LOR's per-request inflight count is a more reliable signal than either EMA rate estimates or filtered latency when queues are deep. (2) At 4 rps baseline this run, PO2 exhibits its known burst variance (P99=4,688 ms), giving MPC and Kalman a spurious short-

TABLE 3. Overload sweep: 8× hetero, 500 req per cell. vs_PO2 column shows MPC-PO2 and Kalman-PO2 mean relative to PO2.

Rate	Policy	Mean (ms)	Stdev	P99 (ms)	vs PO2
4 rps	LOR	**314.4**	203.7	1620	—
	PO2	379.3	560.9	4688	—
	MPC-PO2	360.9	239.9	2082	−4.8%
	Kalman	356.0	223.4	1965	−6.1%
6 rps	LOR	384.9	538.5	3766	—
	PO2	**326.5**	224.1	1914	—
	MPC-PO2	450.5	586.8	4247	+38.0%
	Kalman	380.1	247.7	2042	+16.4%
8 rps	LOR	**416.5**	574.1	4314	—
	PO2	445.9	642.2	4675	—
	MPC-PO2	479.3	600.4	4193	+7.5%
	Kalman	500.9	632.1	4492	+12.3%
12 rps	LOR	**456.6**	637.7	4892	—
	PO2	463.5	609.3	4345	—
	MPC-PO2	541.6	704.1	4602	+16.8%
	Kalman	643.5	755.6	3923	+38.8%

run advantage — further evidence that the short-run 500-request result (§5.1) should not be over-interpreted.

4 Related Work

LLM Serving. PagedAttention [2] and continuous batching [9] established the modern vLLM serving stack. Disaggregated PD architectures [4], [5], [10] separate prefill and decode for independent scaling. Sarathi-Serve [1] addresses prefill-decode interference via chunked prefills.

Load Balancing. PO2 [3] achieves $O(\log \log n)$ maximum queue depth. vLLM-router [8] implements round-robin, PO2, consistent hashing, and cache-aware routing with an approximate radix tree for prefix affinity. Our work shows that naive MPC over queue-depth signals does not improve on PO2, and identifies what richer signals would be needed.

MPC for Systems. MPC [6] with OSQP [7] enables sub-millisecond constrained optimization. Its application to LLM routing is nascent; our study characterizes the conditions under which a queue-depth feedback signal is insufficient.

5 Kalman-PO2: Validating the Diagnosis

The root cause analysis points to a concrete fix: replace queue-depth (which is masked by PO2's implicit balancing) with *observed request latency* as the routing signal. Latency remains discriminative even when queues are balanced, because a capacity-constrained server (`max-num-seqs`= 8) takes longer per request regardless of queue depth — it batches fewer requests per step.

We implement **Kalman-PO2**: a PO2-style router where the server-selection score is the Kalman-filtered estimate of each server's mean request latency, rather than weighted inflight count. The Kalman state per server is:

$$\hat{L}_{k|k} = \hat{L}_{k-1|k-1} + K_k(z_k - \hat{L}_{k-1|k-1}), \quad K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R}$$

where z_k is the observed latency of completed request k, $Q = 400\,\text{ms}^2$ (process noise), $R = 3600\,\text{ms}^2$ (measurement noise), $\hat{L}_0 = 350\,\text{ms}$, and $P_0 = 40{,}000\,\text{ms}^2$ (high initial uncertainty for fast cold start). Routing: sample two servers, pick lower $\hat{L}$. No QP, no arrival rate estimation, no regularization.

5.1 Short-Run Results (500 Requests)

Table 4 compares all three policies at 8× heterogeneity with 500 warm-server requests. Kalman-PO2 achieves lower mean (327 ms vs. 386 ms) and dramatically lower P99 (410 ms vs. 4,527 ms for PO2), and outperforms MPC-PO2 as well.

TABLE 4. 8× heterogeneity, 4 rps, 500 req. Kalman filter converged estimates: fast=309 ms, slow=366 ms.

Policy	Mean (ms)	Stdev (ms)	P99 (ms)
PO2	386.6	583.9	4527.2
MPC-PO2	334.2	44.2	446.2
Kalman-PO2	**327.0**	**36.3**	**410.1**

Log-level analysis reveals the mechanism and its limitations. With only two servers, `random.sample(pool, 2)` always returns both, so routing is deterministic: always pick the server with the lower Kalman estimate. Three phases emerge: (i) *convergence* (req 0–50, 80% fast), (ii) *stable fast-routing* (req 50–300, 100% fast, mean≈309 ms), and (iii) *partial inversion* (req 300–500, ∼64% fast, mean≈330 ms). During phase (iii), concentrating load on the fast server inflates *its* observed latency above the slow server's, causing the Kalman estimate to partially invert. This feedback instability is a fundamental limitation: a latency signal measures *experienced* latency, not *intrinsic* capacity. Kalman-PO2 nonetheless wins over PO2 in this run because PO2's variance is much larger ($\sigma = 584$ ms): PO2 produces 16 requests >1 s (4.4%) and a 4,527 ms P99 from periodic bursts to the slow server.

5.2 Long-Run Behavior (10,000 Requests)

The 10,000-request convergence study (§3, Table 2) reveals the steady-state picture. With a fresh server restart, Kalman-PO2 settles into an equilibrium routing ≈62% of requests to the fast server. This stable partial-routing avoids the burst inversions seen in the short run, but it also eliminates Kalman's advantage: at steady state, mean=342.9 ms vs. PO2=291.8 ms (−17.5%). The gap is indistinguishable from MPC-PO2 (338.2 ms). Both Kalman and MPC remain 15–20% worse than PO2 for the entire 10,000-request run, with no convergence.

The explanation is the same as for MPC: Kalman's latency signal reflects *queue state*, not capacity. Under steady load at 4 rps, the fast server's estimate rises as it absorbs more traffic, and the system finds a sub-optimal equilibrium rather than converging to a true capacity-proportional split.

On homogeneous clusters (1×), Kalman-PO2 incurs a 6.1% overhead (315 ms vs. 297 ms) from the same concentration effect. An uncertainty-gated fallback to PO2 when

$|\hat{L}_a - \hat{L}_b| < \sqrt{P_a + P_b}$ would address both pathologies — left for future work.

6 Conclusions

We show through experiment and repeated replication that the *choice of feedback signal* is the critical variable in LLM request routing. Queue-depth is a sufficient statistic when load is moderate and servers are stationary — PO2 already exploits it optimally, leaving nothing for MPC to improve.

Switching to latency as the feedback signal (Kalman-PO2) yields a transient benefit: in short runs ($\leq$500 requests), Kalman-PO2 eliminates PO2's burst variance (σ: 584 ms$\rightarrow$36 ms) and achieves a 10$\times$ lower P99 (4,527$\rightarrow$410 ms). But the 10,000-request replication reveals that this advantage does not persist: Kalman-PO2 settles at the same $\sim$16% overhead as MPC-PO2. Both signals measure *experienced* rather than *intrinsic* latency, and under sustained load PO2's implicit balancing eliminates any exploitable differential.

Takeaways for practitioners: (1) **PO2 and LOR are robust across all load regimes tested** — from moderate (4 rps) to 3$\times$ overload (12 rps). Adding MPC or Kalman complexity makes things worse, not better, at every scale. (2) **LOR is underrated at high load**: it ties or beats PO2 at 8–12 rps where PO2's random sampling occasionally misdirects requests. (3) **Signal fidelity matters more than controller sophistication**: both MPC's EMA rate estimates and Kalman's filtered latency measure load state rather than intrinsic capacity, making them systematically exploitable by the very routing decisions they drive. (4) A feedback signal that encodes intrinsic capacity directly (e.g. time-to-first-token, or throughput under a controlled probe) is the missing ingredient for any predictive router to outperform PO2.

Open directions: uncertainty-gated fallback to PO2 when Kalman estimates overlap within their posterior variance; TTFT-specific routing for prefill-decode disaggregated topologies where queue depth and latency decouple; and probing-based capacity estimation to break the fundamental signal limitation identified here.

~~Acknowledgments~~ Do Not Include in Submission

Do **not** include acknowledgments in this blind-review version.

References

[1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Sarathi-serve: Efficient LLM inference by piggybacking decodes with chunked prefills. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023.

[3] M. Mitzenmacher. The power of two choices in randomized load balancing. IEEE Transactions on Parallel and Distributed Systems, 12(10):1094–1104, 2001.

[4] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA), 2024.

[5] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.

[6] J. B. Rawlings, D. Q. Mayne, and M. Diehl. Model Predictive Control: Theory, Computation, and Design. Nob Hill Publishing, 2nd edition, 2017.

[7] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. OSQP: An operator splitting solver for quadratic programs. Mathematical Programming Computation, 12:637–672, 2020.

[8] vLLM Team. vLLM production stack and router. In GitHub Repository: vllm-project/vllm, 2024. https://github.com/vllm-project/vllm.

[9] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2022.

[10] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.